Introduction

These Jupyter notebooks provide an introduction to Python for students taking the course **Advanced Robotics Laboratory (00340401)**.

Each notebook includes short exercises to help you practice what you have learned. For every exercise notebook, there is also a solved version. For example:

- `1-Introduction-to-Python-student.ipynb`
- `1-Introduction-to-Python.ipynb` (solved)

Throughout the notebooks, we will work with an imaginary Crazyflie drone. Crazyflie drones are widely used in research, making them a useful example for applying Python concepts. By the end of the notebooks, you will have built a simple trajectory simulation for a Crazyflie drone.

> **Note:** This program is only a simplified example and does not represent a realistic drone simulation.

The notebooks cover the following topics, which are directly related to the other labs:

1. Introduction to Python: basic syntax
2. Matrices and vectors
3. Basic plotting
4. The `class` keyword

Feel free to skip any notebook that seems too easy. If you already have experience programming in Python, such as from a previous Python course, you can likely skip the introductory notebook.

> **Warning:** We recommend that you **do not** use LLMs to solve the exercises. They are optional, and you will learn more by working through them on your own.

Installation

Google Colab (recommended)

The easiest way to run these notebooks is with Google Colab. No installation is required.

1. Open Google Colab.
2. Click **File** → **Open notebook** → **GitHub**.
3. Enter the repository URL.
4. Select the notebook you want to run.
5. Click the play button to execute the cells.

You can also upload a notebook directly to Colab by dragging and dropping the `.ipynb` file.

Running locally

To run the notebooks locally, you will need to:

1. Install Python
2. Install `miniconda`
3. Install the required dependencies

Instructions for Windows, macOS, and Linux are provided below.

Windows

- Go to the Python downloads page.
- Download the correct version for your system, usually **x86-64**.
- Follow the installer steps, and make sure to add Python to your `PATH` (Why?).
- Install `miniconda` by following the instructions here.

macOS/Linux

Python is often already installed by default. We recommend installing `miniconda` so that you can avoid installing packages into your system Python environment and easily work with Jupyter notebooks.

To run these notebooks locally, install Conda.

After installing `conda`, create the environment using the following command after downloading `robotics-lab-intro-environment.yml`:

```
conda env create -f robotics-lab-intro-environment.yml
```

If you are working locally, we also recommend installing Visual Studio Code.

Once the environment has been created, you will be able to run the notebooks using the `robotics-lab-intro` environment.

Dependencies

If you prefer to install the dependencies manually, you will need these two packages:

1. `numpy`
2. `matplotlib`

PDF Version

The PDF version includes the latest solved versions alonside the instructions included in the `README.md`

1-Introduction-to-Python

April 9, 2026

1 Introduction to Python: Basic syntax

Like all languages Python has its own rules and basic syntax. In this notebook we will review the following 4 subjects:

1. Variables
2. Conditionals
3. Functions
4. Loops
5. External packages

These four subjects are crucial in order to be able to write code efficiently and perform most basic tasks.

1.1 Variables

Variables in Python are defined using the `=` sign. Variables can be numbers, strings or other data types. A variable name must start with a letter or underscore, and can contain letters, numbers, and underscores.

[1]:
```
# Example variables of different data types

# Numbers
robot_id = 1
max_speed = 3.0

# Strings
robot_name = "crazyflie"

# Boolean
is_robot_active = False

print(f"Robot Name: {robot_name}, ID: {robot_id}, Max Velocity: {max_speed} [m/
↪s], Active: {is_robot_active}")
```

```
Robot Name: crazyflie, ID: 1, Max Velocity: 3.0 [m/s], Active: False
```

1.1.1 Mission

The Crazyflie's motors have a maximum current limit to protect the battery and motor electronics. In this task, you will use a conditional to ensure that the commanded motor current does not exceed the `max_motor_current`. If the calculated current (`desired_current`) exceeds this limit, you should cap it to the maximum allowed value. This is a safety feature used in real drones.

[2]:
```
max_motor_current = 2.0  # [Amperes] - Maximum allowed motor current
desired_current = 2.5    # [Amperes] - Current command we want to send

print(f"Max Motor Current: {max_motor_current} [A]")
print(f"Desired Current: {desired_current} [A]")

# Limit the current to the maximum if it exceeds the limit
actual_current = None

if desired_current > max_motor_current:
    actual_current = max_motor_current
else:
    actual_current = desired_current

print(f"Actual Motor Current: {actual_current} [A]")
```

```
Max Motor Current: 2.0 [A]
Desired Current: 2.5 [A]
Actual Motor Current: 2.0 [A]
```

2 Functions

Functions are reusable blocks of code that perform specific tasks. They take inputs (parameters) and return outputs, creating a mapping between input and output values. We use functions to make our code easier to understand and more modular. It can also help us avoid repeating code. In Python the keyword `def` is used to define a function, followed by the name

[3]:
```
# Test the safety check functions with the existing max_motor_current variable
def is_current_safe(desired_current, max_current):
    if desired_current > max_current:
        return False
    else:
        return True
def show_current_safety(desired_current, max_motor_current):
    if is_current_safe(desired_current, max_motor_current):
        print("Current is within safe limits.")
    else:
        print("Warning: Desired current exceeds maximum limit!")


desired_current_1 = 1.5  # [Amperes] - A safe current command
```

```
desired_current_2 = 2.5  # [Amperes] - An unsafe current command

show_current_safety(desired_current_1, max_motor_current) # Expected output:␣
↪"Current is within safe limits."
show_current_safety(desired_current_2, max_motor_current) # Expected output:␣
↪"Warning: Desired current exceeds maximum limit!"
```

```
Current is within safe limits.
Warning: Desired current exceeds maximum limit!
```

2.0.1 Mission

Now it is time for you to define your own function. We will slowly start working on our crazyflie simulation. We will start by creating a function `update_dynamics` that updates the position of our robot based on the parameters:

- `current_position_x`: The current position in the `x` axis.
- `velocity_x`: The currrent velocity in the `x` axis.
- `dt`: The simulation timestep.

The function should return:

- `new_position_x`: A number representing the new position in the `x` axis.

[4]:
```
def update_dynamics(current_position_x, velocity_x, dt):
    # Simple dynamics update: new_position = old_position + velocity * time_step
    new_position_x = current_position_x + velocity_x * dt
    return new_position_x


x0 = 0.0  # Initial position in meters
velocity_x = 2.0  # Velocity in meters per second
dt = 1.0  # Time step in seconds

x1 = update_dynamics(x0, velocity_x, dt)  # Position after one time step

print(f"Initial Position: {x0} m")
print(f"Velocity: {velocity_x} m/s")
print(f"Time Step: {dt} s")
print(f"Updated Position: {x1} m") # Expected output: "Updated Position: 2.0 m"
```

```
Initial Position: 0.0 m
Velocity: 2.0 m/s
Time Step: 1.0 s
Updated Position: 2.0 m
```

2.1 Loops

Loops are used to repeat actions efficiently.

- A **`for` loop** is best when the number of iterations is known.

- A **`while` loop** is best when repeating until a condition becomes false.

2.1.1 Mission

Let's use the `update_dynamics` function we created last time (remember that if you couldn't solve it you can check the `1-Introduction-to-Python-SOLVED.ipynb` notebook for a hint). This time we will reduce the `dt` variable to `0.1` and use a `for` loop to simulate `1.0` seconds into the future and update the dynamics by adding velocity.

[5]:
```
# Simulate 1.0 s into the future using smaller time step
dt = 0.1
simulation_time = 1.0
N = int(simulation_time / dt)

current_position_x = x0

for step in range(N):
    current_position_x = update_dynamics(current_position_x, velocity_x, dt)

print(f"Initial position: {x0:.2f} m")
print(f"Velocity: {velocity_x:.2f} m/s")
print(f"dt: {dt:.2f} s, steps: {N}")
print(f"Position after {simulation_time:.2f} s: {current_position_x:.2f} m")
```

```
Initial position: 0.00 m
Velocity: 2.00 m/s
dt: 0.10 s, steps: 10
Position after 1.00 s: 2.00 m
```

2.1.2 Mission

What if we wanted to use a `while` loop in this case? Now instead of using a `for` loop we want the robot to move until it has reached `x1 = 1.0` and print the time it took for the robot to reach that position.

[6]:
```
# Simulate until the position is 1m
current_position_x = x0
t = 0.0

while current_position_x < 1.0:
    current_position_x = update_dynamics(current_position_x, velocity_x, dt)
    t += dt

print(f"Initial position: {x0:.2f} m")
print(f"Velocity: {velocity_x:.2f} m/s")
print(f"dt: {dt:.2f} s")
print(f"Simulated time: {t:.2f} s")
print(f"Position after {t:.2f} s: {current_position_x:.2f} m")
```

```
Initial position: 0.00 m
Velocity: 2.00 m/s
dt: 0.10 s
Simulated time: 0.50 s
Position after 0.50 s: 1.00 m
```

2.2 External packages

External packages let us reuse code written by others instead of building everything from scratch. They provide tested tools for common tasks such as:

- Math and arrays (`numpy` and `math`)
- Data analysis (`pandas`)
- Plotting (`matplotlib`)
- Robotics and simulation libraries

This makes development faster, code cleaner, and projects more reliable.

We install packages with `pip` (Python's package manager), for example:

```
pip install numpy
```

Some packages like `math` are already included with Python. Below you can find examples of importing a module and using its methods. See that the `sin` and `cos` methods take radians as input (like in MATLAB).

[7]:
```
import math

drone_orientation_deg = 90
drone_orientation_rad = math.radians(drone_orientation_deg)
# Simple example: sine and cosine of the drone orientation
sin_orientation = math.sin(drone_orientation_rad)
cos_orientation = math.cos(drone_orientation_rad)

print(f"Orientation: {drone_orientation_deg}° ({drone_orientation_rad:.2f}␣
↪rad)")
print(f"sin(theta) = {sin_orientation:.2f}")
print(f"cos(theta) = {cos_orientation:.2f}")
```

```
Orientation: 90° (1.57 rad)
sin(theta) = 1.00
cos(theta) = 0.00
```

2.2.1 Mission:

Import `numpy` to the notebook and conver the orientation of the drone from degrees to radians using NumPy method `deg2rad`.

[8]:
```
import numpy as np
```

[9]:
```
# Some useful methods for working with robots
drone_orientation_deg = 90
drone_orientation_rad = np.deg2rad(drone_orientation_deg)

print(f"Drone orientation: {drone_orientation_deg} degrees =␣
↪{drone_orientation_rad:.2f} radians")
```

```
Drone orientation: 90 degrees = 1.57 radians
```

2-Matrices-and-vectors

April 9, 2026

1 Matrices and vectors

A **list** in Python is an **ordered, mutable collection** of items.

- **Ordered**: items keep their position (index).
- **Mutable**: you can change it after creation (add, remove, or update elements).
- It can store different data types in the same collection.

[129]:
```
# Demonstrate a Python list
my_list = [1, "apple", 3.14, True]

print("Original list:", my_list)
print("First item:", my_list[0])

# Lists are mutable
my_list.append("new item")
my_list[1] = "orange"

print("Updated list:", my_list)
print("Length:", len(my_list))
```

```
Original list: [1, 'apple', 3.14, True]
First item: 1
Updated list: [1, 'orange', 3.14, True, 'new item']
Length: 5
```

2 NumPy arrays

A **NumPy array** (`ndarray`) is a fast, structured container for numerical data.

- **Homogeneous type**: all elements are stored as the same data type (for example, all integers or all floats).
- **Fixed size**: once created, its size does not change (unlike a Python list).
- **Efficient storage**: data is stored in contiguous memory, making operations faster.
- **Vectorized operations**: arithmetic is applied to all elements at once, without manual loops.
- **Supports dimensions**: arrays can be 1D (vector), 2D (matrix), or higher-dimensional.

Compared to Python lists, NumPy arrays are typically much faster and more convenient for mathematical and scientific computing.

IMPORTANT When working in the lab we will mostly rely on NumPy arrays since they are more efficient and can effectively represent the data that is of interest to us (sensor readings, robot locations, etc.).

See below for different initialization of *NumPy* arrays

[130]:
```
import numpy as np

# Two random 3x2 arrays and their sum
A = np.random.rand(3, 2)
B = np.random.rand(3, 2)

print("A:\n", A)
print("B:\n", B)

# Two 3x1 unit vectors
x = np.array([1.0, 0.0, 0.0])
y = np.array([0.0, 1.0, 0.0])
```

```
A:
 [[0.06335977 0.44373609]
 [0.18965705 0.75062001]
 [0.36508588 0.81793543]]
B:
 [[0.10862835 0.99417722]
 [0.39419335 0.53842622]
 [0.5454175  0.84897384]]
```

Now we will use the arrays to demonstrate common operations:

[131]:
```
# Addition
C = A + B
print("A + B:\n", C)

# Element-wise operations
alpha_1 = 2.0
A_scaled = alpha_1 * A
print(f"\n{alpha_1} * A:\n", A_scaled)

# Dot and cross products
dot_xy = x.T @ y
cross_xy = np.cross(x, y)

print("\nx:\n", x)
print("y:\n", y)
print("dot(x, y):", dot_xy)
print("cross(x, y):\n", cross_xy)
```

```
A + B:
 [[0.17198812 1.4379133 ]
```

```
 [0.58385039 1.28904624]
 [0.91050338 1.66690928]]

2.0 * A:
 [[0.12671954 0.88747218]
 [0.3793141  1.50124003]
 [0.73017176 1.63587087]]

x:
 [1. 0. 0.]
y:
 [0. 1. 0.]
dot(x, y): 0.0
cross(x, y):
 [0. 0. 1.]
```

2.1 Shape, indexing, and : slicing

Every NumPy array has a `.shape` attribute that returns `(rows, cols)` for a 2D array — **row index always comes first**.

Elements are accessed with zero-based indices: `M[row, col]`.
The colon `:` means *all elements along that axis*: - `M[:, 0]` — every row, column 0 (the entire first column) - `M[0, :]` — row 0, every column (the entire first row)

[132]:
```python
M = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9],
], dtype=float)

print("M:\n", M)
print("Shape of M:", M.shape)       # (3, 3): 3 rows, 3 columns
print("M[0, 2]:", M[0, 2])          # row 0, column 2
print("First column M[:, 0]:", M[:, 0])   # all rows, column 0
print("Last row   M[2, :]:",   M[2, :])   # row 2, all columns
```

```
M:
 [[1. 2. 3.]
 [4. 5. 6.]
 [7. 8. 9.]]
Shape of M: (3, 3)
M[0, 2]: 3.0
First column M[:, 0]: [1. 4. 7.]
Last row   M[2, :]: [7. 8. 9.]
```

2.1.1 Mission

There is a neat connection between colon indexing and matrix-vector multiplication: multiplying a matrix by a **standard basis vector** e_i — a vector with a 1 at position i and 0s elsewhere — extracts the i-th column, giving the same result as `M[:, i]`.

$$M\, e_i = M[:, i]$$

Using the matrix `M` defined above: 1. Extract the **third column** (index 2) using `:` indexing. 2. Build the standard basis vector `e3 = [0, 0, 1]` and compute `M @ e2`. 3. Verify both results are equal using `np.allclose`.

> Similarly, we can also extract rows by multiplying with a transposed unit vector from the left:
>
> $$e_i^\top M = M[i, :]$$

[133]:
```
# 1. Third column via colon indexing
col_indexing = M[:, 2]

# 2. Third column via matrix-vector product with standard basis vector
e3 = np.array([0, 0, 1])
col_matvec = M @ e3

print("Third column via indexing:    ", col_indexing)
print("Third column via M @ e3:      ", col_matvec)
print("Are they equal?", np.allclose(col_indexing, col_matvec))
```

```
Third column via indexing:     [3. 6. 9.]
Third column via M @ e3:       [3. 6. 9.]
Are they equal? True
```

2.1.2 Mission

1. Extract the **third row** (index 2) using `:` indexing.
2. Use the same `e2 = [0, 0, 1]` and compute `e2 @ M`.
3. Verify both results are equal using `np.allclose`.

[134]:
```
# 1. Third row via colon indexing
row_indexing = M[2, :]
# 2. Third row via vector-matrix product with standard basis vector
e2 = np.array([0, 0, 1])
row_matvec = e2 @ M
print("Third row via indexing:       ", row_indexing)
print("Third row via e2 @ M:        ", row_matvec)
print("Are they equal?", np.allclose(row_indexing, row_matvec))
```

```
Third row via indexing:        [7. 8. 9.]
Third row via e2 @ M:         [7. 8. 9.]
Are they equal? True
```

2.1.3 Mission

As you have seen in the examples above the scalar product and addition operators are defined by `*` and `+`. Let's use our function from the previous notebook `update_dynamics` but now instead of passing a position or velocity in `x` we will pass NumPy arrays representing both dimensions. Use a loop to move for `1.0` seconds from `(0.0,0.0,0.0)` with a velocity of `(1.0, 0.5,0.0)`. You should: - Store the positions at every time step into an array called `positions`. - Store the timesteps into an array called `timesteps`. Use the `np.linspace` method to create it. - Your end position should be `(1,0.5)` - You should use NumPy arrays not lists or tuples

`np.linspace(start, stop, num)` creates a NumPy array of `num` evenly spaced values from `start` to `stop` (including both endpoints by default).

It is useful because it:

- Generates **uniform time grids** (for example, simulation timesteps).
- Avoids manually computing step values in loops.
- Returns a **NumPy array**, so it works directly with vectorized math and plotting.
- Makes code clearer and less error-prone when you need a fixed number of samples (like positions over `1.0` second).

[135]:
```
# Copying function definition
def update_dynamics(current_position_x, velocity_x, dt):
    # Simple dynamics update: new_position = old_position + velocity * time_step
    new_position_x = current_position_x + velocity_x * dt
    return new_position_x

x0 = np.array([0.0, 0.0, 0.0])  # Initial position
v = np.array([1.0, 0.5, 0.0])   # Velocity in x and y
dt = 0.1                     # Time step in seconds

t_end = 1.0
N = int(t_end / dt)
positions = np.zeros((N, 3))  # To store positions at each time
times = np.linspace(0, t_end, N)  # Time array for plotting
positions[0] = x0

for i in range(N):
    positions[i] = update_dynamics(positions[i-1], v, dt)
print("\nPositions over time:\n", positions)
```

```
Positions over time:
 [[0.1  0.05 0.  ]
 [0.2  0.1  0.  ]
 [0.3  0.15 0.  ]
 [0.4  0.2  0.  ]
 [0.5  0.25 0.  ]
 [0.6  0.3  0.  ]
 [0.7  0.35 0.  ]
```

```
[0.8  0.4  0.  ]
[0.9  0.45 0.  ]
[1.   0.5  0.  ]]
```

2.1.4 Homogeneous transformation matrix (quick reminder)

In robotics, we often want to represent **rotation and translation together** in one matrix.
A homogeneous transform does exactly that:

$$\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix}$$

- $\mathbf{R}$: rotation matrix
- $\mathbf{t}$: translation vector

For a 3D pose ($SE(3)$):

$$\mathbf{T} = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

For planar robots ($SE(2)$, 2D motion), we use:

$$\mathbf{T_{2D}} = \begin{bmatrix} \cos\theta & -\sin\theta & t_x \\ \sin\theta & \cos\theta & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

Useful properties:

1. **Composition** (chain transforms):

$$\mathbf{T}_{ac} = \mathbf{T}_{ab}\mathbf{T}_{bc}$$

2. **Inverse**:

$$\mathbf{T}^{-1} = \begin{bmatrix} \mathbf{R}^\top & -\mathbf{R}^\top\mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix}$$

This is why homogeneous matrices are central in robotics: they provide a compact way to move between coordinate frames.

2.1.5 Example: rotating a point into a drone's body frame

Suppose our Crazyflie is sitting at the **origin** but has rotated **45°** (yaw).
A landmark is at `[1, 0]` in the world frame — 1 m along the world x-axis.
Where does that landmark appear relative to the drone?

We build the SE(2) transform `T_world_drone` (the drone's pose in the world frame), invert it to get `T_drone_world`, then apply it to the landmark.

[136]:
```
import numpy as np

# Drone pose: yaw = 45°, no translation (sitting at origin)
yaw = np.deg2rad(45)
tx, ty = 0.0, 0.0

T_world_drone = np.array([
    [np.cos(yaw), -np.sin(yaw), tx],
    [np.sin(yaw),  np.cos(yaw), ty],
    [0,            0,           1 ]
])

# Landmark in world frame (homogeneous coordinates)
p_world = np.array([1.0, 0.0, 1.0])

# Inverse: world frame → drone body frame
T_drone_world = np.linalg.inv(T_world_drone)
p_body = T_drone_world @ p_world

print("Drone yaw: 45°, at origin")
print("Landmark in world frame:     ", p_world[:2])
print("Landmark in drone body frame:", np.round(p_body[:2], 4))
# Result: [cos(-45°), sin(-45°)]   [0.707, -0.707]
# The landmark is ahead-right in the drone's rotated frame
```

```
Drone yaw: 45°, at origin
Landmark in world frame:      [1. 0.]
Landmark in drone body frame: [ 0.7071 -0.7071]
```

2.1.6 Mission

A Crazyflie drone is flying in the world frame. It has landed at position `[0.5, 0.5]` (in metres) and rotated to a yaw of **90°**. A landmark is located at `[1.0, 0.0]` in the world frame.

Your task: 1. Build the SE(2) homogeneous transform `T_world_drone` that describes the drone's pose in the world frame (yaw = 90°, translation = [0.5, 0.5]). 2. Express the landmark `p_world = [1.0, 0.0]` in **homogeneous coordinates** (append a 1). 3. Compute the inverse transform `T_drone_world` and use it with the `@` operator to find where the landmark appears in the drone's body frame.

Hints: `np.cos`, `np.sin`, `np.deg2rad`, `np.linalg.inv`, `@`

[137]:
```
import numpy as np

# Drone pose in world frame: yaw = 90 degrees, position = [0.5, 0.5]
yaw = np.deg2rad(90)
```

```python
tx, ty = 0.5, 0.5

# SE(2) homogeneous transform: T_world_drone
T_world_drone = np.array([
    [np.cos(yaw), -np.sin(yaw), tx],
    [np.sin(yaw),  np.cos(yaw), ty],
    [0,            0,           1 ]
])

# Landmark position in world frame (homogeneous coordinates)
p_world = np.array([1.0, 0.0, 1.0])

# Express landmark in drone body frame
T_drone_world = np.linalg.inv(T_world_drone)
p_body = T_drone_world @ p_world

print("T_world_drone =\n", np.round(T_world_drone, 4))
print("Landmark in world frame:      ", p_world[:2])
print("Landmark in drone body frame: ", np.round(p_body[:2], 4))
# Expected: [-0.5, -0.5]
# The drone faces +y at 90° yaw, so the landmark (world x=1) is
# 0.5 m behind the drone and 0.5 m to its right in body frame.
```

```
T_world_drone =
 [[ 0.  -1.   0.5]
 [ 1.   0.   0.5]
 [ 0.   0.   1. ]]
Landmark in world frame:       [1. 0.]
Landmark in drone body frame:  [-0.5 -0.5]
```

3-Basic-plotting

April 9, 2026

1 Basic plotting

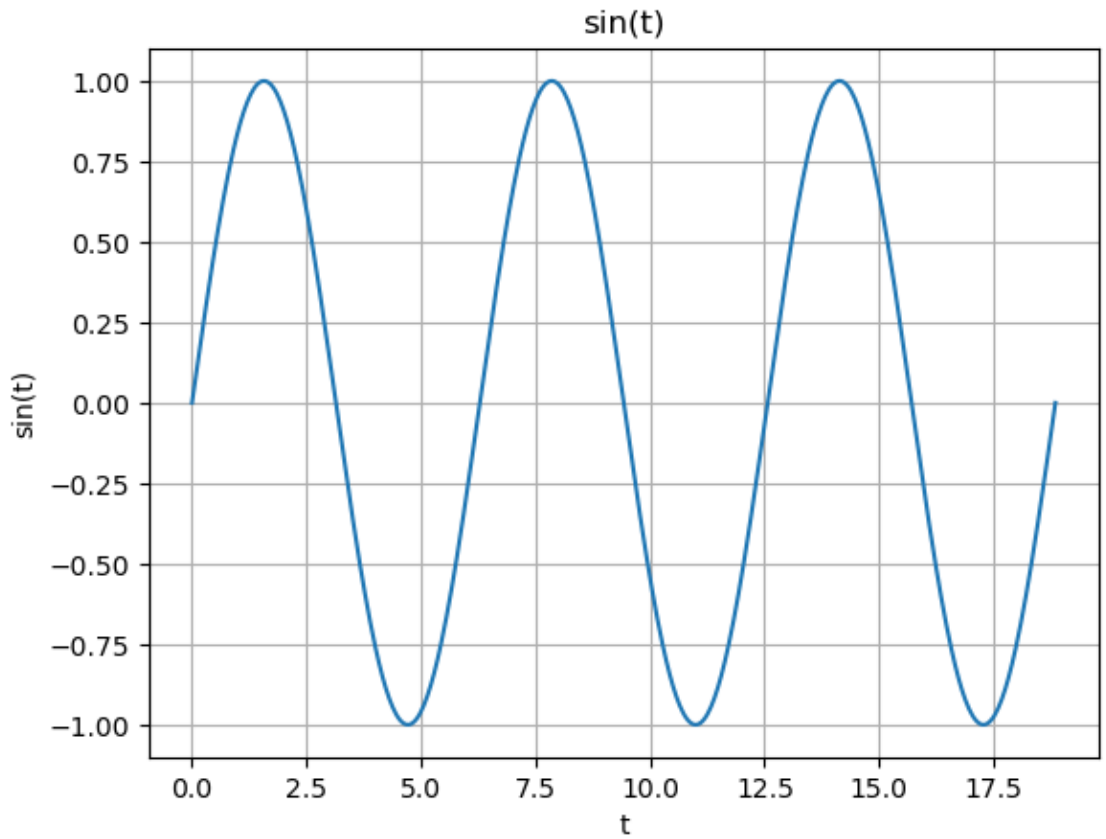
In this notebook, we will use **Matplotlib** (typically `matplotlib.pyplot`) for plotting.
It is a standard Python visualization library that is flexible, widely supported, and works well in Jupyter notebooks.

Why is this important? Plotting our lab results is a key to be able to communicat ethem clearly clearly.

1.1 Examples

We will demonstrate the usage of `matplotlib` below. You will mostly work with the `plot` and `scatter` command. You can use the `title`, `xlabel`, `ylabel` and other commands to nicely format your plots.

[9]:
```
# Basic example of plotting a sine wave using NumPy and Matplotlib
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 6 * np.pi, 500)
y = np.sin(t)

plt.plot(t, y)
plt.title("sin(t)")
plt.xlabel("t")
plt.ylabel("sin(t)")
plt.grid(True)
plt.show()
```



1.1.1 Mission

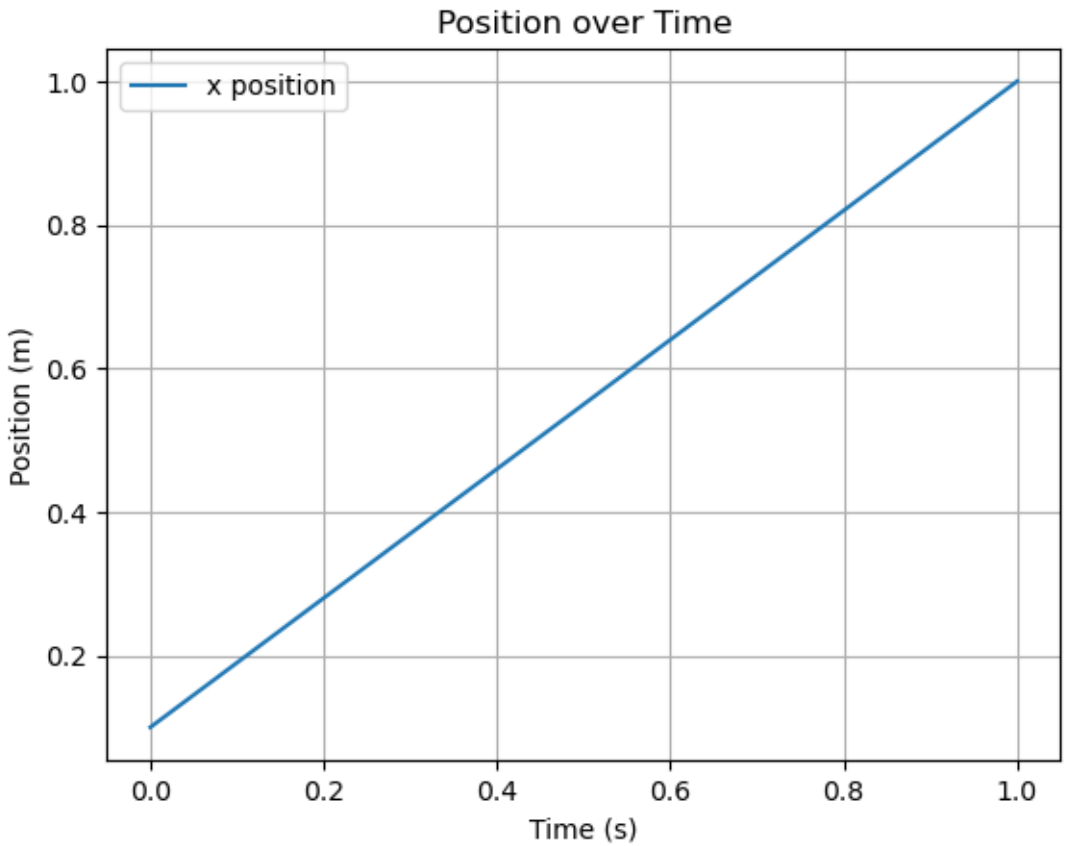
We will use the code from lab 2 to simulate a drone flying for one second with a given velocity. You will have to make the following plots: 1. $x(t)$ 2. $y(t)$ 3. $x - y$ projection. Add a red dot for the start position and a green one for the end position.

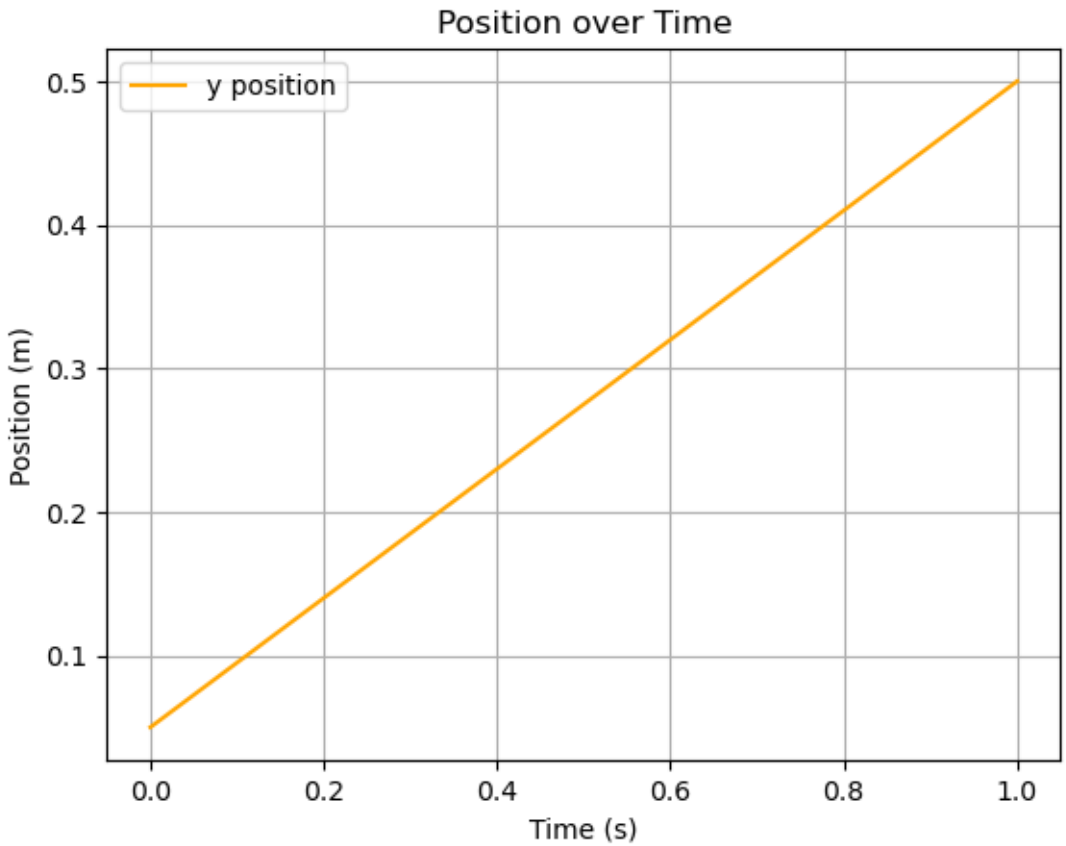
Use the `plot` method. Make sure to label your axis and add a title to your plot.

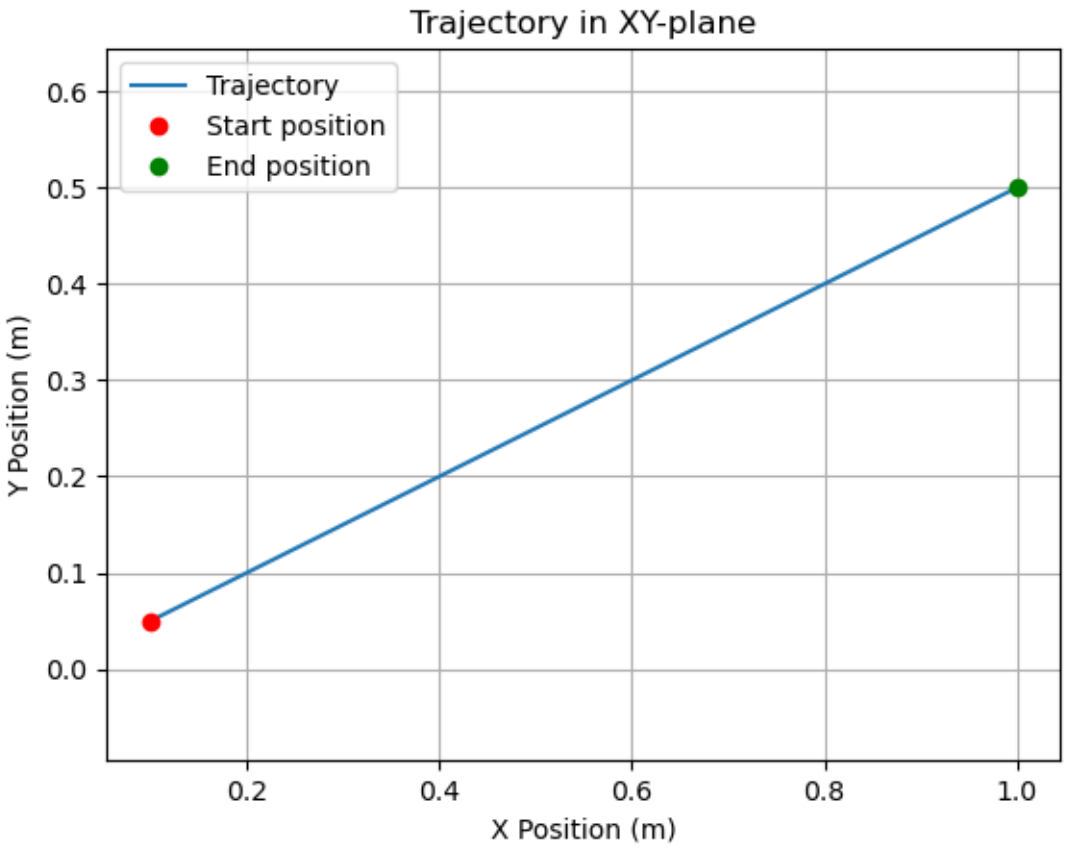
[10]:
```
# From 2-Matrices-and-vectors.ipynb
def update_dynamics(current_position_x, velocity_x, dt):
    # Simple dynamics update: new_position = old_position + velocity * time_step
    new_position_x = current_position_x + velocity_x * dt
    return new_position_x

x0 = np.array([0.0, 0.0,0.0])  # Initial position 2D
v = np.array([1.0, 0.5,0.0])   # Velocity in x and y
dt = 0.1                      # Time step in seconds

t_end = 1.0
N = int(t_end / dt)
positions = np.zeros((N, 3))  # To store positions at each time
```

```
times = np.linspace(0, t_end, N)  # Time array for plotting
positions[0] = x0

for i in range(N):
    positions[i] = update_dynamics(positions[i-1], v, dt)
```

[11]:
```
# Plotting the trajectory in the x-axis as a function of time
plt.figure()
plt.plot(times, positions[:, 0], label='x position')
plt.title("Position over Time")
plt.xlabel("Time (s)")
plt.ylabel("Position (m)")
plt.grid(True)
plt.legend()
plt.show()

# Plotting the trajectory in the y-axis as a function of time
plt.figure()
plt.plot(times, positions[:, 1], label='y position', color='orange')
plt.title("Position over Time")
plt.xlabel("Time (s)")
plt.ylabel("Position (m)")
plt.grid(True)
plt.legend()

# Plotting the trajectory in the xy-plane
plt.figure()
plt.plot(positions[:, 0], positions[:, 1], label='Trajectory')
plt.plot(positions[0, 0], positions[0, 1], 'ro', label='Start position')
plt.plot(positions[-1, 0], positions[-1, 1], 'go', label='End position')
plt.title("Trajectory in XY-plane")
plt.xlabel("X Position (m)")
plt.ylabel("Y Position (m)")
plt.grid(True)
plt.legend()
plt.axis('equal')  # Set equal aspect ratio for x and y axes
plt.show()
```


Position over Time
x position
1.0
0.8
0.6
0.4
0.2
Position (m)
0.0
0.2
0.4
0.6
0.8
1.0
Time (s)


Position over Time
y position
0.5
0.4
0.3
0.2
0.1
Position (m)
0.0
0.2
0.4
0.6
0.8
1.0
Time (s)


Trajectory in XY-plane
Trajectory
Start position
End position
0.6
0.5
0.4
0.3
0.2
0.1
0.0
Y Position (m)
0.2
0.4
0.6
0.8
1.0
X Position (m)

4-The-class-keyword

April 9, 2026

1 The `class` keyword

A **class** is a way to group related data and behavior under one concept.

An **attribute** is a piece of information stored inside an object. It represents the object's state at a given moment.

A **method** is a function that belongs to a class. It defines what the object can do and how its state can be read or changed.

An **object** is a concrete instance of a class with its own attribute values.

Why is this important? Throughout the lab you will meet classes in most labs. Being familiar with both the concept behind it but also the syntax quirks will give you an advantage.

1.1 The `drone` class

We will use the `Drone` class below to show you how a class works. We will re-use most of the `update_dynamics` method and modify it to include yaw logic.

[82]:
```
import numpy as np

class Drone:
    def __init__(self, x0, name, yaw0=0.0):
        """
        x0   : initial position as a numpy array, e.g. np.array([0.0, 0.0, 0.0])
        name : string identifier for the drone
        yaw0 : initial yaw angle in radians (default 0.0)
        """
        self.name = name
        self.position = x0.copy()
        self.yaw = yaw0
        self.trajectory = [x0.copy()]  # stores all positions
        self.yaw_history = [yaw0]      # stores yaw at each timestep
        self.timesteps = [0.0]         # stores corresponding time values

    def update_dynamics(self, velocity,  yaw,dt):
        """
        Advance the drone one step forward in time.
```

```
        velocity : numpy array with the same shape as x0
        dt       : simulation timestep in seconds
        yaw      : yaw angle in radians for this timestep
        """
        self.position = self.position + velocity * dt
        self.yaw = yaw
        self.trajectory.append(self.position.copy())
        self.yaw_history.append(self.yaw)
        self.timesteps.append(self.timesteps[-1] + dt)
```

1.2 Understanding `self` and `__init__`

- `self` represents the current object instance.
- Inside a method, writing `self.position` means "the `position` attribute of this specific object."
- Python passes `self` automatically when you call a method with `object.method(...)`.

1.2.1 The `__init__` method

- `__init__` is the initialization method of a class.
- It runs automatically when a new object is created, for example with `Drone(x0, name)`.
- Use it to define the initial state of the object (attributes such as `name`, `position`, and logs like `trajectory`).

1.2.2 Important class syntax details

- Every instance method must have `self` as its first parameter.
- Attributes that belong to each object should be stored as `self.attribute_name`.
- Methods can both read and modify object state, which is how behavior is attached to data.

1.2.3 Mission

Try it out. Create a `Drone`, give it a constant velocity, and simulate a few steps.

[83]:
```
x0 = np.array([0.0, 0.0, 0.0])   # start at the origin
v  = np.array([1.0, 0.5, 0.0])   # constant velocity (m/s)
dt = 0.1                          # timestep (s)

drone = Drone(x0=x0, name="cf1")  # create a Drone instance with x0 and a name␣
 ↪of your choice

for _ in range(10):
    drone.update_dynamics(v, 0.0, dt)

print(f"Drone: {drone.name}")
print(f"Final position : {drone.position}")
print(f"Trajectory ({len(drone.trajectory)} points):")
for t, pos in zip(drone.timesteps, drone.trajectory):
    print(f"  t={t:.1f}s  pos={pos}")
```

```
Drone: cf1
Final position : [1.  0.5 0. ]
Trajectory (11 points):
  t=0.0s  pos=[0. 0. 0.]
  t=0.1s  pos=[0.1  0.05 0.  ]
  t=0.2s  pos=[0.2 0.1 0. ]
  t=0.3s  pos=[0.3  0.15 0.  ]
  t=0.4s  pos=[0.4 0.2 0. ]
  t=0.5s  pos=[0.5  0.25 0.  ]
  t=0.6s  pos=[0.6 0.3 0. ]
  t=0.7s  pos=[0.7  0.35 0.  ]
  t=0.8s  pos=[0.8 0.4 0. ]
  t=0.9s  pos=[0.9  0.45 0.  ]
  t=1.0s  pos=[1.  0.5 0. ]
```

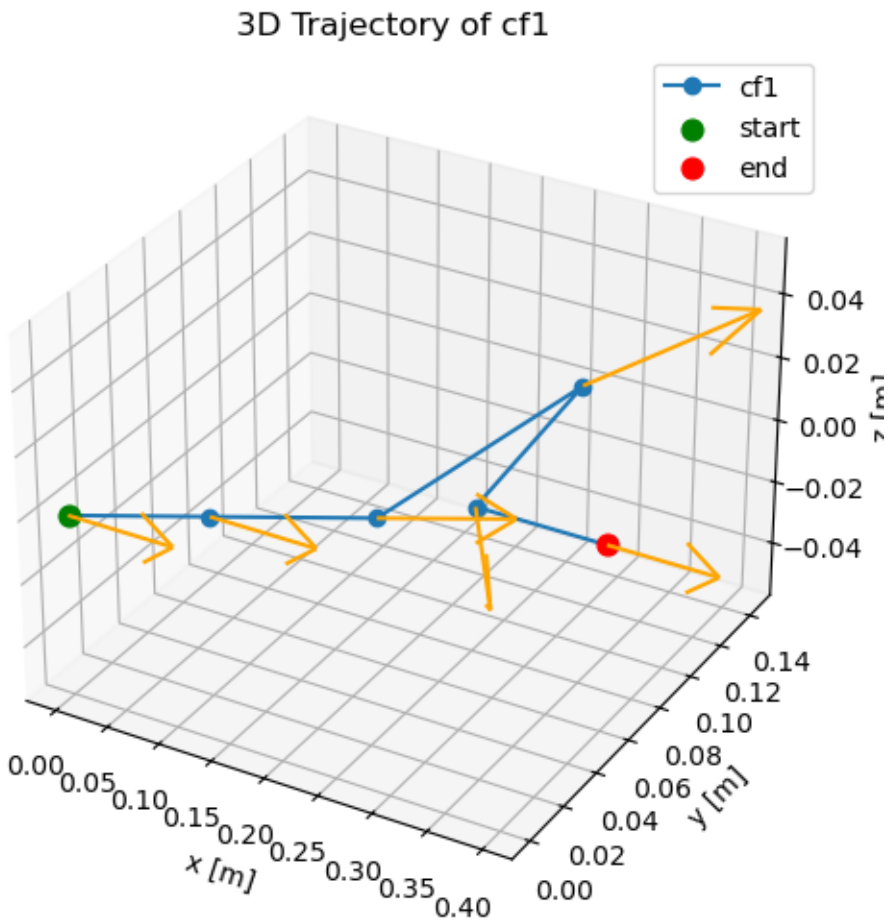
1.2.4 Mission

We will now create our "simulation" for the Crazyflie drone. We will write a class `CrazyFlieSim` together. We will provide some helper functions and some of the structure. Your tasks will be: - Pass and store required initialization parameters (`max_speed`, `dt`, `drone`) to the `__init__` method. - Initialize the missing class attributes. - Define a `simulate` method that: - Takes a NumPy array of `velocity_commands` - Saturates the velocity command so that the drone never exceeds `max_speed` - Updates the drone's position using the `update_dynamics` method with this timesteps' command and the defined `dt` in the `__init__` method

[84]:
```python
import numpy as np
import matplotlib.pyplot as plt


class CrazyFlieSim:
    def __init__(self, max_speed, dt, drone):
        self.max_speed = float(max_speed)
        self.dt = float(dt)
        self.drone = drone

    def simulate(self, velocity_commands, yaw_commands):
        """Simulate a sequence of velocity and yaw commands.

        velocity_commands : iterable of 3D velocity vectors (m/s)
        yaw_commands      : iterable of yaw angles in radians, one per velocity␣
 ↪command
        """
        for v_cmd, yaw in zip(velocity_commands, yaw_commands):
            v_cmd = np.asarray(v_cmd, dtype=float)
            speed = np.linalg.norm(v_cmd)

            if speed > self.max_speed and speed > 0:
                v_cmd = (v_cmd / speed) * self.max_speed
```

```python
            self.drone.update_dynamics(v_cmd, float(yaw), self.dt)

    def plot_trajectory(self, arrow_scale=0.1):
        """Plot 3D trajectory, start/end points, and yaw arrows.

        arrow_scale : length of each yaw arrow in metres (default 0.1).
        """
        traj = np.array(self.drone.trajectory)
        yaws = np.array(self.drone.yaw_history)

        fig = plt.figure(figsize=(7, 5))
        ax = fig.add_subplot(111, projection="3d")
        ax.plot(traj[:, 0], traj[:, 1], traj[:, 2], marker="o", label=self.
 ↪drone.name)
        ax.scatter(traj[0, 0], traj[0, 1], traj[0, 2], color="green", s=60,␣
 ↪label="start")
        ax.scatter(traj[-1, 0], traj[-1, 1], traj[-1, 2], color="red", s=60,␣
 ↪label="end")

        # Draw yaw arrows as a single vectorized quiver call.
        ax.quiver(traj[:, 0], traj[:, 1], traj[:, 2],
                  arrow_scale * np.cos(yaws),
                  arrow_scale * np.sin(yaws),
                  np.zeros_like(yaws),
                  normalize=False, color="orange", arrow_length_ratio=0.3)

        ax.set_xlabel("x [m]")
        ax.set_ylabel("y [m]")
        ax.set_zlabel("z [m]")
        ax.set_title(f"3D Trajectory of {self.drone.name}")
        ax.legend()
        plt.tight_layout()
        plt.show()


# Example usage
x0 = np.array([0.0, 0.0, 0.0])
drone = Drone(x0=x0, name="cf1")
sim = CrazyFlieSim(max_speed=1.2, dt=0.1, drone=drone)
velocity_commands = [
    [1.0, 0.2, 0.0],
    [1.5, 0.3, 0.0],
    [0.4, 1.0, 0.0],
    [0.2, -0.8, 0.0],
    [1.7, 0.0, 0.0],
]
```

```
yaw_commands = [0.0, 0.2, 0.8, -0.5, 0.0]

sim.simulate(velocity_commands, yaw_commands)
sim.plot_trajectory(arrow_scale=0.1)
```



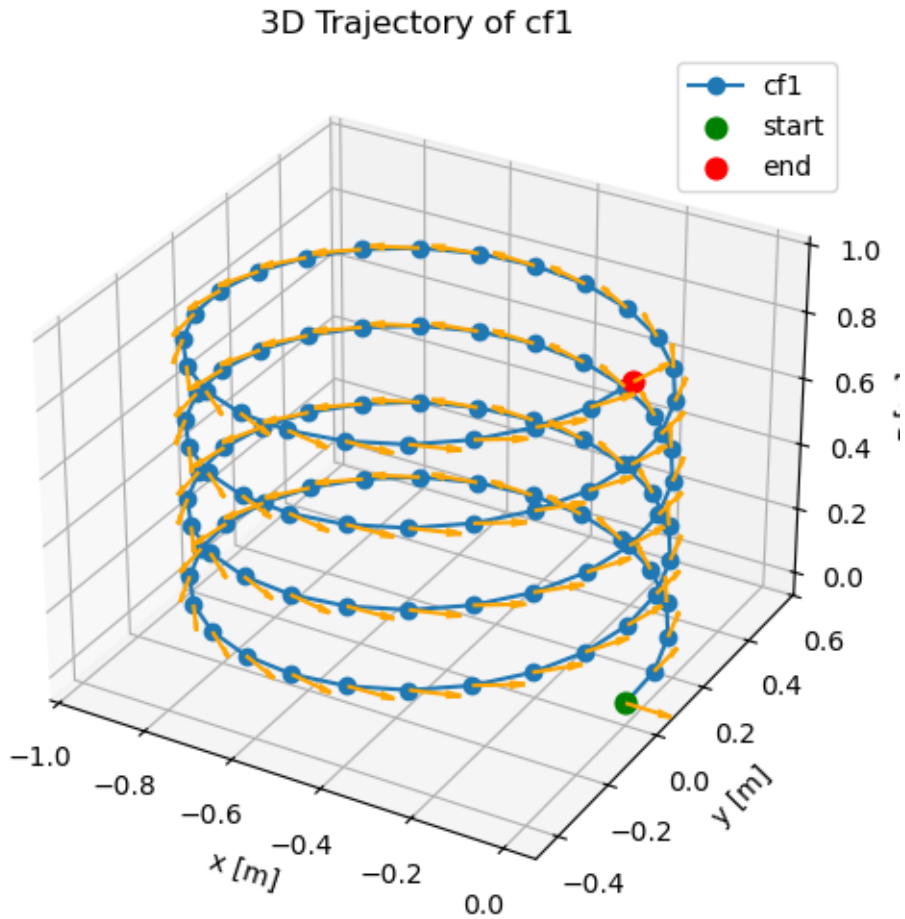
1.2.5 Mission

Now use NumPy to define 100 velocity commands so that the simulated drone follows an upwards spiral trajectory. To help you we provide a parametrized form of a spiral where r is the radius of the spiral, v_z is the velocity upwards and t is the time.

$$\mathbf{x}(\mathbf{t}) = [r * \cos(t), r\sin(t), v_z \cdot t]$$
$$\mathbf{x}'(\mathbf{t}) = [-r * \sin(t), r\cos(t), v_z]$$

Now use that array of velocity commands to simulate a new trajectory. drone = Drone(x0=x0, name="cf1") sim = CrazyFlieSim(max_speed=1.2, dt=0.1, drone=drone)

[85]:
```
drone = Drone(x0=x0, name="cf1")
sim = CrazyFlieSim(max_speed=1.2, dt=0.1, drone=drone)

# Spiral parameters
r = 1.0          # radius [m]
vz = 0.2         # upward velocity [m/s]
f_hz = 0.4    # spiral frequency [Hz]
num_commands = 100
dt = sim.dt      # simulation time step [s]

# Time vector (one command per simulation step)
t = np.arange(num_commands) * dt
omega = 2.0 * np.pi * f_hz #TODO: compute angular velocity from frequency

theta = omega * t
velocity_commands = np.column_stack((
    -r * omega * np.sin(theta),
    r * omega * np.cos(theta),
    np.full_like(t, vz)
))


# Yaw follows the tangent direction of the spiral. Provided for you, but feel␣
↪free to modify if you want to experiment with different yaw profiles.
yaw_commands = np.arctan2(r * omega * np.cos(theta), -r * omega * np.sin(theta))

sim.simulate(velocity_commands, yaw_commands)
sim.plot_trajectory(arrow_scale=0.1)
```


3D Trajectory of cf1
cf1
start
end
1.0
0.8
0.6
0.4
0.2
0.0
−1.0
−0.8
−0.6
−0.4
−0.2
0.0
x [m]
0.6
0.4
0.2
0.0
−0.2
−0.4
y [m]